Fall Recovery of Bipedal Locomotion via Modular Task Decomposition and Force-Decay Curriculum

Codrin Crismariu, Jesse Flores

January 20, 2026

1 Introduction and Problem Statement

Deep Reinforcement Learning (DRL) often struggles with the high-dimensional complexity of full 3D dynamics, leading to slow convergence and poor sample efficiency. As seen in the standard Unity ML-Agents(Juliani et al. [2018]) WalkerAgent implementation, the agent must learn to control 40 continuous actions (joint rotations and strengths) based on a state vector of over 150 features (relative physics for all 16 body parts).

A critical limitation of current baselines is their inability to handle multi-modal behaviors – specifically, the distinct dynamic requirements of steady-state locomotion versus fall recovery. Standard PPO models often fail to learn recovery behaviors because the exploration penalty for failing is too high, leading the agent to adopt conservative policies that avoid failing but never learn to stand back up.

This project proposes to address a limitation of the standard Unity Walker example and investigate a curriculum learning approach to solve those limitations. We hypothesize that separating the control problem into two distinct "expert" networks – one for walking and one for recovery – and training them via a curriculum can lead to the agent developing a policy capable of quick recovery for strategic control that is otherwise impossible to train or discover in one go.

2 Background and Related Work

2.1 Deep Reinforcement Learning in Robotics

The application of DRL, particularly Proximal Policy Optimization (PPO), has achieved major successes in continuous control for robotics. One example would be the work of Heess et al. [2017]. However, data efficiency, training time and stability remain primary concerns, especially in high-dimensional state and action spaces like those found in humanoid locomotion. One of these bottlenecks seen in the Walker example of the Unity ML-agents can be seen when we test

how the agent manages fall recovery. This turns out to be non-existent even for long training time, since the behaviour needed from a humanoid to recover from a fall is too complex.

2.2 Curriculum Learning and Domain Transfer

Curriculum learning (presented in Narvekar et al. [2020]), a method inspired by human learning, involves training a model on progressively harder tasks. Our approach aims to implement two methods, namely Force-Decay training and Modular Task Decomposition. This force-decay method is the most important part of our curriculum because it gradually guides the agent towards the necessary moves to make the robot stand up from a fall.

3 Technical Approach

3.1 Environment and Agent Model

- **Environment:** Unity game engine's ML-Agents Toolkit.
- **Robot:** The standard WalkerAgent robot, which features 16 distinct body parts and a 3D physics-based ArticulationBody skeleton.

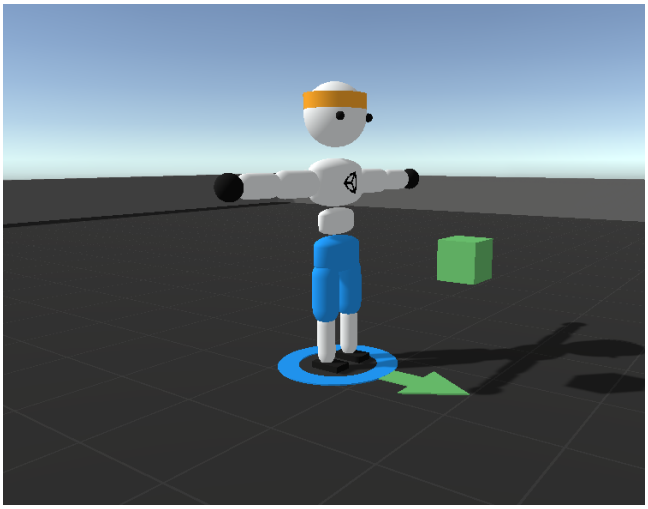


Figure 1: The WalkerAgent robot in its Unity ML-Agents test environment.

3.2 State Space (S)

The state vector remains identical for all training stages. The observation collection method provides a high-dimensional vector with approximately 150 features, which includes:

- **Root/Torso:** Relative velocity, goal velocity, rotation deltas to target, and target position.
- **Per-Body-Part (16 parts):** Ground contact, relative linear velocity, relative angular velocity, relative position to hips, local rotation, and current joint strength.

3.3 Action Space (A)

The action space is a 40-dimensional continuous action space. This vector controls:

- **PD Target Rotations:** The target rotation for the joint.
- **Joint Strength:** The stiffness/strength of the PD controllers (defined as springiness and damper in the unity environment).

This high-dimensional action space makes exploration difficult and strongly motivates a curriculum.

3.4 Reward Function (R)

We will use the baseline multiplicative reward function $R_{\text{velocity}} \cdot R_{\text{look-direction}}$.

$$R_{\text{baseline}} = R_{\text{velocity-tracking}} \cdot R_{\text{look-direction}}$$

- $R_{\text{velocity-tracking}}$: A sigmoidal reward for matching the target velocity.
- $R_{\text{look-direction}}$: Based on the dot product of the head's forward vector and the target direction.

Along with this, for our curriculum we also apply a reward for our recovery curriculum based on the agents head height squared and it's alignment with a vector pointing up. This reward is supposed to reward the agent for standing up. The head height is squared to further incentivize the agent to jump to stand up instead of just maintaining a height at which it gets rewards.

$$R_{\text{curriculum}} = R_{\text{head-height}} + R_{\text{up-right}}$$

- $R_{\text{head-height}}$ Based on the height of the agents head relative to the floor
- $R_{\text{up-right}}$: Based on the dot product of the hips position and a vector pointing up

3.5 Learning Algorithm: Proximal Policy Optimization (PPO)

We will use Proximal Policy Optimization (PPO as presented in Schulman et al. [2017]) as our DRL algorithm, as it is the state-of-the-art, on-policy algorithm used by ML-Agents for complex continuous control.

PPO operates as an Actor-Critic method, meaning it uses two separate neural networks:

- **The Actor (Policy Network):** This network takes the agent's state (the 150+ feature observation vector) as input and is responsible for handling the 40-dimensional continuous action space. It does not output a discrete action. Instead, it outputs the parameters for a stochastic policy. Specifically, for each of the 40 actions, the network outputs a mean (μ) and a standard deviation (σ). These parameters define 40 independent Gaussian probability distributions, one for each joint's rotation and strength. The final action vector is then sampled from these distributions: $a_i \sim \mathcal{N}(\mu_i, \sigma_i)$ for each action i. This stochastic sampling is PPO's mechanism for exploration. The Actor's job is to learn to shift the means (μ) of these distributions toward optimal values and tune the standard deviations (σ) to balance exploration and exploitation.
- **The Critic (Value Network):** This network also takes the state as input but outputs a single scalar value, $V(s)$. This value represents its estimate of the total future discounted reward the agent can expect to get from its current state. The Critic's job is to learn how good the current state is.

How PPO Learns

PPO collects a batch of experience (trajectories) using its current Actor policy. It then updates its networks for several epochs using this data. The update is driven by the Advantage Function, $A(s, a)$, which is calculated using the Critic's value estimates:

$$A(s, a) \simeq (R_t + \gamma V(s_{t+1})) - V(s_t)$$

This $A(s, a)$ value represents "how much better or worse" the action a was than the Critic's initial expectation. (this was initially explored in Schulman et al. [2015])

- If $A(s, a)$ is positive, PPO updates the Actor network to increase the log-probability of sampling action a in state s.
- For a Gaussian distribution, this means shifting the mean (μ) closer to the value a that was sampled.
- If $A(s, a)$ is negative, PPO updates the Actor network to decrease the log-probability of sampling action a in state s.

PPO's key innovation is its clipped surrogate objective function. This objective clips the ratio between the new policy and the old policy, preventing the policy update from being too large in a single step. This clipping ensures stability, which is critical for a high-dimensional physics agent where a single bad update could cause the policy to collapse (e.g., learning to fall over immediately).

3.6 Method 1: External Force Decay ("All-in-one" agent)

This method trains a single agent to stand and walk simultaneously by assisting it with an external force. In this curriculum, we apply a force pointed up to the agent's hips pulling it up proportionally to its weight. We apply an external vertical force F_{assist} to the agent to counteract gravity:

$$F_{assist} = k \cdot m \cdot g \tag{1}$$

Where k is the assist coefficient. The curriculum follows a discrete decay plan starting from 80% of the agent's total weight, ending with 0% in 20% decrements. The goal is that this external force guides the agent towards standing up and also allows it to explore different gaits without immediate termination from falling.

3.7 Method 2: Modular Task Decomposition (The "Experts")

This method aims to decouple the problem into two distinct policies, recognising that "walking" and "standing up" are fundamentally different control tasks requiring distinct dynamics. This approach is similar to the one presented in Karlsson [1994]. We approach this problem by placing the two specialized policies to get managed by a high-level machine.

- **The Walker Expert**: Trained without a curriculum, but each episode ends whenever the agent falls. It focuses purely on locomotion and matching the desired velocity without falling without concerning itself on what happens when it falls.
- **The Recovery Expert**: Trained with a curriculum specifically to stand up. It turns out standing up needs a behaviour with a higher learning curve than normal locomotion. For this agent the same curriculum as in the first method is applied but this time the only reward the agent receives is for standing up
- **Switching Logic**: During evaluation, a supervisor algorithm dynamically switches control between the two policies based on the agent's torso deviation angle. For our tests we activated the recovery logic when the angle between the hips and the ground was smaller than 30 degrees and reactivated the walker logic when the angle was bigger than 60. This 30 degree buffer gives the agent the chance to fully recover before starting walking and to refrain from using the recovery logic until fully fallen.

4 Experiments and Evaluation

4.1 Evaluation Methodology and Baselines

To rigorously evaluate the contributions of this paper, we conducted two distinct comparisons:

1. **Curriculum:** We compare the Force-Decay Agent (Method 1) against an agent trained to walk without a curriculum.

2. **Architecture:** We compare the Two-Expert Agent (Method 2) against an agent trained to walk without a curriculum.

All the methods were trained for 10 Million training steps using the ml-agents Unity framework with checkpoints saved every 500k steps. After the training we measured two metrics, how many targets the agent collected over 5000 timesteps and how much of the time did the agent spend not in contact with the ground. (the second one being our measure for the ability to recover from a fall)

4.2 Method 1 ("All-in-one" Agent)

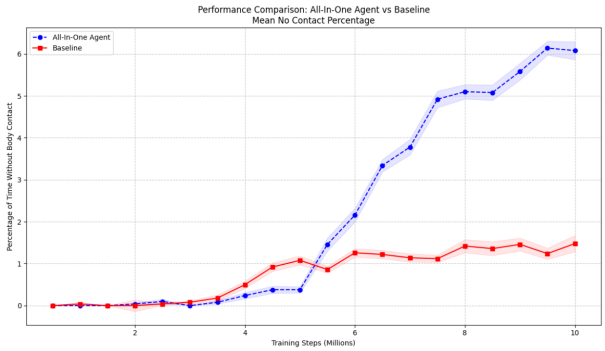


Figure 2: Mean number of targets collected over 5000 timesteps averaged over 100 agents

We clearly see that using our recovery curriculum the agent not only learns to move towards the targets but also learns to recover from falls and continue moving towards the target. The only drawback of this approach is that even though the agent does move towards the target, it does not do this in a way similar to humans. It seems that depending on some randomness in the training it discovers way to jump on his hands and feet at the same time which certainly solves our lack of recovery but also lacks the movement we would like to see from a humanoid robot.

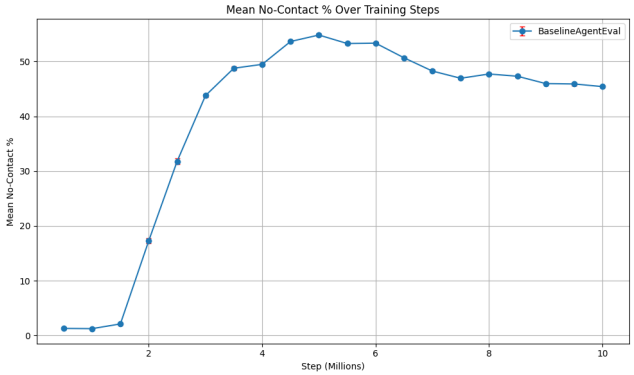


Figure 3: Percentage of time spent without being in contact with the ground

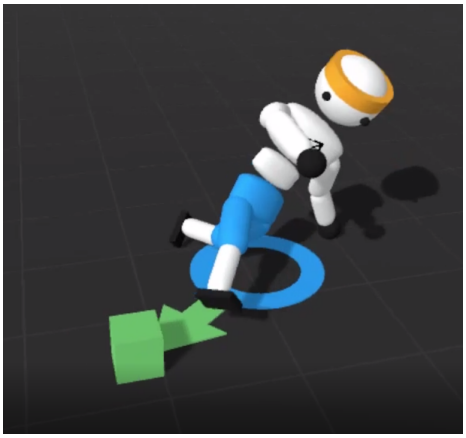


Figure 4: All in one agent performing an uncommon "sideways" walk

4.3 Method 2 (Two Expert Agent)

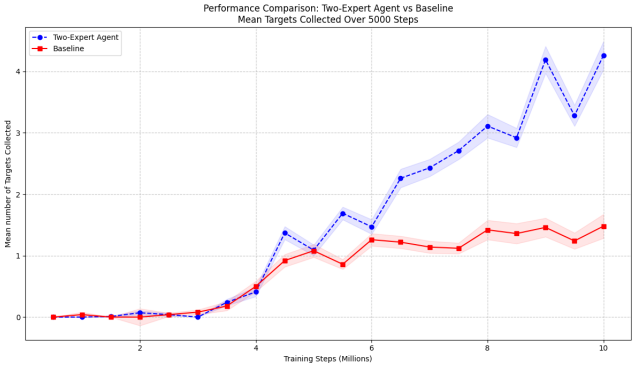


Figure 5: Mean number of targets collected over 5000 timesteps averaged over 100 agents

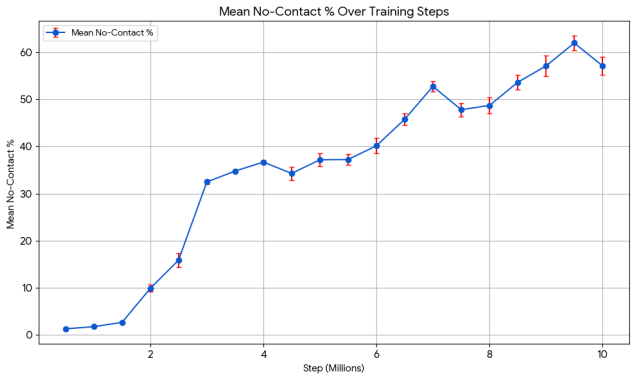
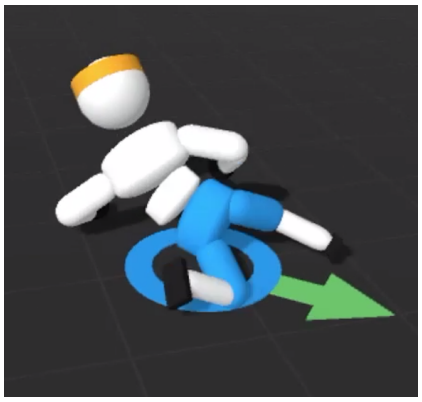


Figure 6: Percentage of time spent without being in contact with the ground

One interesting thing to notice is that the two expert agent does no necessarily perform better at collecting targets compared to the initial method, but it's still way better than the baseline. The key difference is that it still maintains a "human" walk while spending less time on the ground trying to recover after the necessary training. And given that both agents utilized the same Force-Decay curriculum, the performance gap can be attributed to the separation of tasks ("walk" and "recovery").



(a) Fallen agent pushing off the ground

(b) Agent fully recovered from the fall

A significant finding was the emergent human-like recovery strategies of the recovery expert. The standard baseline agent fails its legs and inches its body ineffectively towards the targets in the scene when fallen – unable to lift the torso back up.

In contrast, the two-expert agent discoverd a "push-up" strategy. It learned to plant its hands firmly on the ground, push against the floor to generate leverage, and transition the torso to an upright position once again – a strategy mimicking early human motor learning.

5 Conclusion

This project demonstrates that high-dimensional humanoid control problems are best solved not by deeper networks but by better task definitions. By decomposing complex problems into smaller ones, we can achieve more robust agents with greater capabilities which behave closer to human-like behavior. Curriculum learning showed early signs of good performance while the emergence of arm-based leverage strategies validated the hypothesis that curriculum learning coupled with control management structures can help agents escape local optima in complex physical environments.

References

Nicolas Heess, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, A. Eslami, M. Riedmiller, and D. Silver. Emergence of locomotion behaviours in rich environments. *arXiv preprint arXiv:1707.02286*, 2017. URL `https://arxiv.org/abs/1707.02286`.

Arthur Juliani, Vincent-Pierre Berges, Erdogdu Teng, Thomas Cohen, James Worrall, Yufeng Hwong, Kyla Tma, Dominic D'Alleva, Praneet Goy, Yul and... Hwong, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2018. URL `https://arxiv.org/abs/1809.02627`.

Jonas Karlsson. Task decomposition in reinforcement learning. In *Proceedings of the 1994 AAAI Spring Symposium on Goal-Driven Learning*, AAAI Technical Report SS-94-02, Stanford, CA, 1994. AAAI Press. URL `https://cdn.aaai.org/Symposia/Spring/1994/SS-94-02/SS94-02-006.pdf`.

Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020. URL `http://jmlr.org/papers/v21/20-212.html`.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015. URL `https://arxiv.org/abs/1506.02438`.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL `https://arxiv.org/abs/1707.06347`.